



Appunti di Crittografia (2025/26)

Stefano Zanolli - Magistrale di Ingegneria e Scienze Informatiche

Argomenti:

1. INTRODUZIONE

- Teoria dei numeri
 - COLLEGA GLI ARGOMENTI
-

2. CRITTOGRAFIA ASIMMETRICA

- Crittografia Asimmetrica
- One Way Trapdoor Functions
- Diffie-Hellman - asimmetrico (PKDH)
- RSA

- COLLEGA GLI ARGOMENTI
 - Firma digitale e Hash
-

3. PRG e CRITTOGRAFIA SIMMETRICA

- crittografia simmetrica
 - PRG (Pseudo Random Generators)
 - XOR
 - Cifratura simmetrica a blocchi
 - COLLEGA GLI ARGOMENTI
-

4. SICUREZZA di una PRG

- Teorema di Yao (Next-bit test)
 - HCP (Hard-Core predicate)
 - Generatore di Goldwasser-Micali (BSS)
 - COLLEGA GLI ARGOMENTI
-

5. INTEGRITY del CIPHERTEXT

- PRF (Pseudo Random Functions) e MAC
 - COLLEGA GLI ARGOMENTI
-

6. USI "CREATIVI" E UTILI DELLA CRITTOGRAFIA

- Oblivious transfer
 - Bit commitment
 - Secret sharing (Shamir)
 - Dining cryptographers
 - ZKP (Zero knowledge proofs)
-

1. INTRODUZIONE:

Teoria dei numeri (parte 1)

TEORIA DEI NUMERI

La crittografia utilizza un'algebra particolare $\rightarrow$ l'ALGEBRA MODULARE

$$\mathbb{Z}_n = \{[0], [1], [2], \dots, [n-1]\}$$

classi di equivalenza

partizione

$\mathbb{Z}_n$ è l'insieme dei resti della divisione per n . È ciclico!

esempio: $\mathbb{Z}_6 = \{[0], [1], [2], [3], [4], [5]\}$

in ogni classe di equivalenza ci sono tutti i numeri il cui " $a \% n = \text{classe } []$ "

per esempio $[1] = 1, 7, 13, \dots$ perché $2/6$ dà resto 1
 $\downarrow \quad \downarrow \quad \downarrow$
 $1 \% 6 = 1 \quad 7 \% 6 = 1 \quad 13 \% 6 = 1$
 $\rightarrow$ si scrive $7 \pmod{6} = 1$

Quando usiamo questi insiemi, ci piace avere diverse proprietà (commutativa, etc.)
MA SOPRATTUTTO l'elemento INVERSO. (inverso $\rightarrow$ cifare/decifare)

Per ottenerlo usiamo un insieme più specifico, $\mathbb{Z}_p$ (dove p è un numero primo).

- PERCHÉ?

Utilizziamo "l'identità di Bézout":

se $\text{MCD}(a, p) = 1$,

(un numero
dell'insieme $\mathbb{Z}_p$)

allora:

$$ax + py = 1$$

!! All'interno di $\mathbb{Z}_p$ sono tutti coprimi con p
TRANNE lo zero! ~~[0]~~

- QUINDI:

C'è un ultimo ostacolo, dobbiamo togliere lo zero (classe $[0]$)
perché $\mathbb{Z}_p$ è vischioso da usare.

$$- py = 0 \leftarrow (p \pmod{p} = 0)$$

$\downarrow$
 $7 \% 7 = 0$

$\mathbb{Z}_p^*$, $\mathbb{Z}_n^*$ = Questi insiemi sono composti solo da numeri che possiedono inverso e coprimi con il modulo p/n .

$$\Rightarrow ax = 1$$

$$\Rightarrow x = \frac{1}{a}$$

ESISTE UN INVERSO
PER OGNI $a \neq 0$

$$\mathbb{Z}_n^* = \mathbb{Z}_n \text{ tenendo solo i coprimi con } n.$$

$$\mathbb{Z}_p^* = \mathbb{Z}_p \setminus [0]$$

GENERATORI

$\mathbb{Z}_p^*$ contiene delle classi speciali, i generatori, ovvero dei "motori" capaci di "generare" tutti i numeri dell'insieme. $\{g^1, g^2, g^3, \dots, g^{p-1}\} = \mathbb{Z}_p^*$

esempi:

$$\mathbb{Z}_7^* = \{1, 2, 3, 4, 5, 6\}$$

un generatore è 3:

$$3^1 \pmod{7} = 3$$

$$3^2 \pmod{7} = 2$$

$$3^3 \pmod{7} = 6 \quad \checkmark$$

$$3^4 \pmod{7} = 4$$

$$3^5 \pmod{7} = 5$$

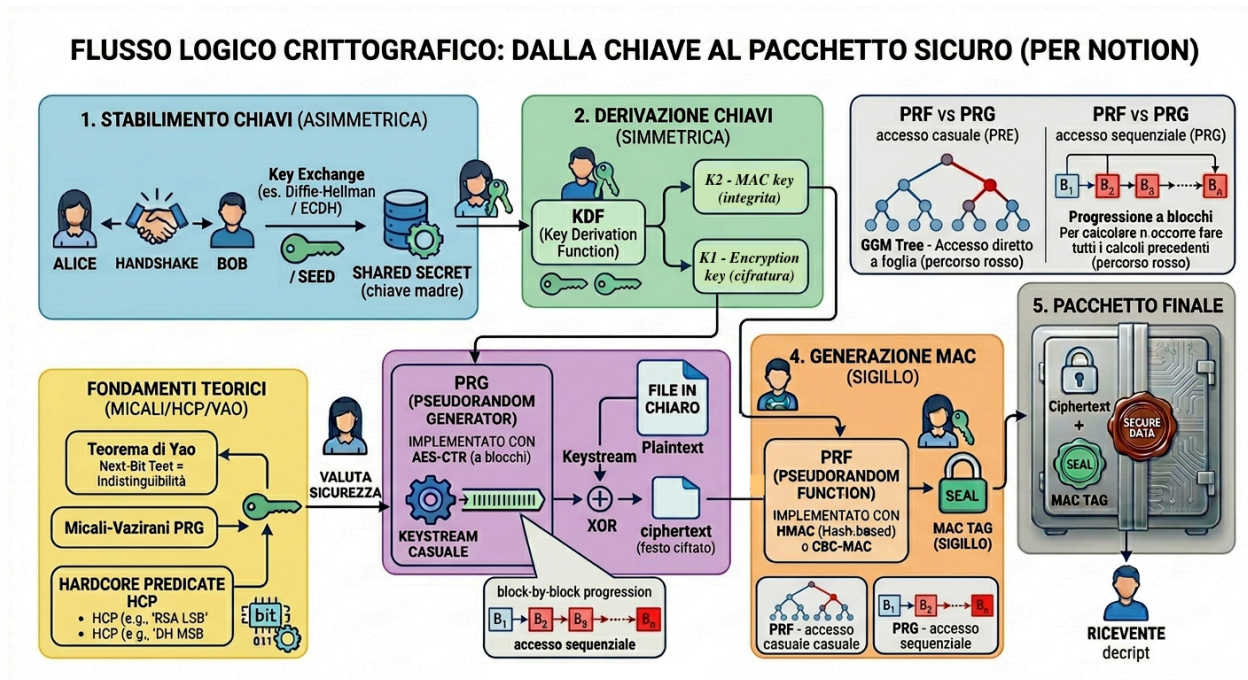
$$3^6 \pmod{7} = 1$$

QUINDI $\rightarrow$ se prendi un generatore hai la certezza che dopo $p-1$ passi, avrai "toccato" tutti gli elementi di $\mathbb{Z}_p^*$.

È su questo che si basa una delle One-Way Trapdoor Function utilizzate nella crittografia, ovvero il LOGARITMO DISCRETO. ($g^x = h \pmod{p}$)

!! $\Rightarrow$ se p ha 600 cifre, anche conoscendo h e g , a causa dell'imprevedibilità dell'algebra modulare, l'unico modo di trovare x è provarle tutte. Questo è impossibile computazionalmente perché ci si metterebbe miliardi di anni. (600 cifre $\Rightarrow p$ è circa 10^{600} . Per prospettiva, atomi dell'universo: 10^{80})

COLLEGA GLI ARGOMENTI (Preview completa)



È prematuro mostrare quest'immagine, ma è utile per dare contesto.

Utilizzerò quest'immagine componendola durante gli appunti un blocchetto alla volta in modo da non perdere mai di vista come ogni argomento si collega al suo successivo.

Teoria dei numeri (parte 2)

Introduzione teorica prima di affrontare la Crittografia Asimmetrica:

Quadrati e Simboli: Legendre vs Jacobi

In $\mathbb{Z}_p^*$, esattamente la metà dei numeri sono "Quadrati".

(residui quadratici)

Nella matematica normale $\sqrt{9} = \pm 3$, $\rightarrow$ 2 radici

nell'algebra modulare: $\sqrt{9 \pmod{11}} = +3$

$$-3 \rightarrow (11-3) = 8$$

$$\exists x \mid x^2 \equiv a \pmod{p}$$

$\rightarrow$ 2 radici

$\Rightarrow$ se per ogni radice ho 2 numeri del gruppo (es: 3 e 8), vuol

dire che solo metà dei numeri sono quadrati, in quanto per ogni quadrato ho, di partenza, una coppia di numeri.

es non vale: $\left(\{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\} \pmod{11} \right)$

$\begin{array}{ccccccccc} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 \\ \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow \\ & + & + & + & + & + & + & + & + & + \\ & & & & 5 & & & & & \end{array}$

5, la metà!

SIMBOLO DI LEGENDRE $\left(\frac{a}{p}\right)$

quando p è un modulo primo, questo "simbolo", ti dice istantaneamente se un numero a è un quadrato.

- Calcolo: si usa il Criterio di Eulero $\rightarrow a^{\frac{p-1}{2}} \pmod{p}$

$\downarrow$
 $+1$: a è un quadrato.
 -1 : a non è un quadrato

SIMBOLO DI JACOBI $\left(\frac{a}{n}\right)$

È l'estensione di Legendre per moduli composti (es. $n = p \times q$).

- Calcolo: si calcola "Spezzando" nei singoli ~~moduli~~ Simboli di Legendre dei suoi fattori $\rightarrow \left(\frac{a}{n}\right) = \left(\frac{a}{p}\right) \times \left(\frac{a}{q}\right)$

$$\overbrace{\left(\frac{a}{p}\right) \times \left(\frac{a}{q}\right)}^{\text{Legendre}} = \overbrace{\left(\frac{a}{n}\right)}^{\text{Jacobi}}$$

$$\begin{array}{lcl} 1. & 1 & \times 1 = 1 \\ 2. & -1 & \times -1 = 1 \end{array} \left. \vphantom{\begin{array}{l} 1. \\ 2. \end{array}} \right\} \text{AMBIGUITÀ}$$

$$\begin{array}{lcl} 3. & 1 & \times -1 = -1 \\ 4. & -1 & \times 1 = -1 \end{array} \left. \vphantom{\begin{array}{l} 3. \\ 4. \end{array}} \right\} \text{Non } n \text{ QUADRATO}$$

$\Rightarrow$ Se Jacobi dà -1 , sei sicuro che n non è un quadrato.
 Se dà $+1$, è ambiguo: potrebbe essere un "falso positivo",
 per saperlo DEVO conoscere la FATTORIZZAZIONE di n (cioè $p \cdot q$).

facendo un successivo passo, possiamo creare altri 4 combinazioni

Jacobi		Jacobi	
Quadrato	$\times$	Quadrato	= Quadrato
Falso Quadrato	$\times$	Falso Quadrato	= Quadrato
Quadrato	$\times$	Falso Quadrato	= FALSO QUADRATO!!

$\hookrightarrow$

Qui SEMBRANO TUTTI dei quadrati $\Rightarrow$ Simbolo di Jacobi $= 1$,
 ma non tutti lo sono davvero se guardiamo Legendre!!

Su questo si basa il Critto-Sistema di Micali.

IMPORTANTE: Quando diciamo $n = p \times q$ intendiamo un n composto
 che deriva da 2 numeri primi p e q .

MA COME SI CALCOLA LEGENDRE?

All'occhio abbiamo solo p o q , mentre il simbolo richiede anche
 un numero a ($\exists_p$).

$\Rightarrow$ L'2 È IL TUO MESSAGGIO!!!!

in $\mathbb{Z}_n$, sapere se il mio messaggio è quadrato o meno è
 impossibile senza scomporre il problema chiedendosi se

a è quadrato su p e poi su q .

$$n = \underbrace{\left(\frac{a}{p}\right) \times \left(\frac{a}{q}\right)}_{\text{Simb. Jacobi}} \neq !!$$

Introduciamo quindi gli altri mostri sacri delle funzioni One-Way Trapdoor più usate e famose in crittografia:

- FATTORIZZAZIONE $\rightarrow$ dato un n composto ($n = p \times q$), trovare proprio p e q è impossibile computazionalmente a livello di tempo, perché dovrei provare tutti i divisori a mano.

- ESTRAZIONE DELLA RADICE QUADRATA $\rightarrow$

dato un messaggio $a \in \mathbb{Z}_n$, come trovo se a è un quadrato oppure no?

Anche questo è impossibile senza conoscere p e q per fare il simbolo di Jacobi (a/n) (per moduli composti) e quindi senza saper fattorizzare.

- RICONOSCERE QUADRATI E FALSI QUADRATI: $\rightarrow$

Come spiegato prima, entrambi questi dall'esterno appaiono come normali quadrati.

Ma anche qui senza conoscere i simboli di Jacobi, e quindi saper fattorizzare, e quindi conoscere p e q , è impossibile distinguerli.

2. CRITTOGRAFIA ASIMMETRICA

Invece di affrontare gli argomenti in blocco unendo crittografia Simmetrica e poi Asimmetrica, ho scelto di andare seguendo uno schema un pò più logico, facendomi la seguente domanda:

" Se la crittografia si basa su una chiave, a prescindere da come venga usata, il PRIMO ostacolo vero è capire come questa chiave può essere scambiata tra 2 o più persone su internet no? "

La risposta ci conduce al primo argomento vero del corso: La Crittografia Asimmetrica.

DEFINIZIONE PRO E CONTRO:

La Crittografia Asimmetrica funziona con 2 chiavi che hanno ruoli distinti:

- una chiave PUBBLICA viene usata dagli altri per **cifrare** nei miei confronti,
- l'altra è PRIVATA (segreta) e serve ad invertire il processo, ovvero **decifrare** i messaggi che mi arrivano.

PRO:

- Risolvono il problema dello scambio della chiave su internet (Tutti utilizzano le chiavi pubbliche per cifrare e private per decifrare i propri messaggi senza bisogno di scambi).
- Introducono altri vantaggi che poi vedremo come la Firma Digitale per garantire proprietà fondamentali della sicurezza come "AUTHENTICITY" e "INTEGRITY".

CONTRO:

- Il difetto più grande è la loro lentezza computazionale, se dovessimo usarle per cifrare e decifrare ogni messaggio, Internet si pianterebbe. Per questo il loro utilizzo si limita quasi sempre a creare un tunnel sicuro in cui scambiare la VERA chiave di cifratura, la chiave simmetrica.

- Inoltre algoritmi come RSA hanno problemi di padding e prevedibilità su messaggi brevi, e problemi di malleabilità (Malleability attack).

ONE-WAY TRAPDOOR FUNCTIONS

Le One-Way Trapdoor Functions sono operazioni semplicissime nell'andare in una direzione,

one-way, ma computazionalmente impossibili nel tornare indietro, così impossibili da basare la maggior parte della nostra sicurezza oggi giorno su di esse.

Le abbiamo già viste ma le rielencherò qui sotto in modo schematico:

Problema	Formula / Definizione	Intrattabilità (Difficoltà)
1. LOGARITMO DISCRETO	$g^x \equiv h \pmod{p}$	Conoscendo la base g e il risultato h , è impossibile trovare l'esponente x se p ha centinaia di cifre.
2. FATTORIZZAZIONE	$n = p \times q$	Dato un numero enorme n , è impossibile risalire ai due fattori primi p e q che lo hanno generato.
3. RADICE QUADRATA	$x^2 \equiv a \pmod{n}$	Dato a , stabilire se esiste un x tale che elevato al quadrato dia a . È difficile quanto fattorizzare n .
4. INDISTINGUIBILITÀ (QRP)	$Vero \times Falso = Falso$	Senza conoscere i fattori di n , è impossibile distinguere un "vero quadrato" da un "falso quadrato" (pseudo-residuo).

DIFFIE-HELLMAN (PKDH)

DIFFIE HELLMAN (PKDH)

- Scopo = scambiare una chiave simmetrica in un ambiente poco fidato come Internet.

- Come funziona =

1. Si stabilisce un'algebra $\mathbb{Z}_p^*$;
2. Si stabilisce un generatore g ;
3. Sia Alice che Bob scelgono il loro segreto :

Alice $\rightarrow a$, Bob $\rightarrow b$

4. Entrambi elevano il loro segreto al generatore g , e se lo inviano:

Alice $\rightarrow g^a = A(\text{mod } p)$ $\xrightarrow{A, g, p}$ Bob

Bob $\rightarrow g^b = B(\text{mod } p)$ $\xrightarrow{B, g, p}$ Alice

5. Infine elevano il loro segreto al numero ricevuto, generando la stessa K (chiave).

Alice $\rightarrow B^a = (g^b)^a \rightarrow K$
||| (Master Key)

Bob $\rightarrow A^b = (g^a)^b \rightarrow K$

PERCHÉ È SICURO? $\rightarrow$ Logarithm Discreto ($g^a = A(\text{mod } p)$)

Anche se non è DIMOSTRABILMENTE sicuro, si chiama ipotesi DH.

Note: I **gruppi Diffie-Hellman** sono set di parametri matematici (q, α) standardizzati e testati per garantire la massima robustezza contro il calcolo dei logaritmi discreti. Usare parametri fissi assicura l'interoperabilità tra dispositivi e permette ai **firewall** di convalidare istantaneamente il livello di sicurezza della connessione senza bloccarla.

RSA

RSA

- Scopo = utilizza la coppia di chiavi pubblica e privata per scambiarsi informazioni segrete su internet. Chiude, usando il lucchetto pubblico del destinatario, il messaggio, garantendo che solo lui con la sua chiave privata possa aprirlo.

- Come funziona:

1. Scelgo due numeri primi p e q SEGRETI e li moltiplico ottenendo il mio modulo composto n (con $n = p \times q$).

2. Utilizzo la funzione di Eulero per trovare un nuovo modulo segreto dato dal numero di COPRIMI in $\mathbb{Z}_n$.

$$\phi(n) = (p-1) \times (q-1) \rightarrow \mathbb{Z}_{\phi(n)}$$

3. A questo punto, per far funzionare cifratura/decifratura cerchiamo due numeri che mi facciano fare il giro completo dell'orologio segreto ($\mathbb{Z}_{\phi(n)}$):

$$(m^e)^d \equiv m \pmod{n}, \quad e \cdot d \equiv 1 \pmod{\phi(n)}$$

h. risultati:

(di solito un numero primo di default es: 3)

- chiave pubblica $\Rightarrow (e, n)$

$$\text{CIFRO: } (\text{messaggio})^e = C$$

- chiave privata $\Rightarrow (d)$

$$\text{DECIFRO: } C^d = ((\text{messaggio})^e)^d = \text{messaggio}_{\text{PLAIN}}$$

- Come si genera d ? $\rightarrow$ ALGORITMO DI EUCLIDE ESTESO



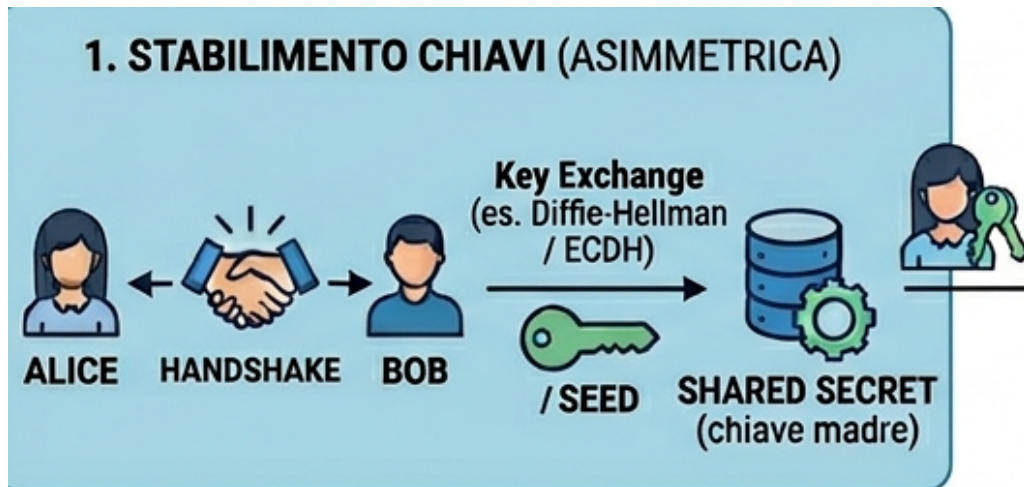
- Sia per creare la chiave privata d

- Sia usato da un hacker per ottenerla se scoprisse $\phi(n)$.

- Perché RSA è debole nella pratica?

- soffre messaggi brevi (padding per risolvere)
- È deterministico quindi possibilmente prevedibile
- È malleabile ($m^e \times \text{hacker-}m^e \rightarrow \bar{e}$ un messaggio alterato valido)

COLLEGA GLI ARGOMENTI



Ecco che piazziamo il primo blocco, quello dello Scambio della chiave da cui partiremo per fare la vera cifratura di un messaggio e molto altro ancora.

Aggiungo qui sotto altri 2 argomenti importanti, la Firma Digitale e la funzione di Hash.

FIRMA DIGITALE e HASH

FIRMA DIGITALE e HASH.

- Scopo della firma digitale = garantire INTEGRITY, AUTHENTICATION e NON-REPUDIATION del file che firmo.

$$\text{firma} = (m)^d$$

Ovvero serve a dimostrare matematicamente CHI ha davvero inviato il file e a garantire che il suo contenuto non sia stato modificato durante il tragitto.

- problema di pesantezza = inviare (messaggio, firma) è pesante perché è come inviare il doppio del messaggio originale.

- Soluzione → HASH = che cos'è un Hash?

↓
in crittografia, il compito di una funzione Hash è prendere un file di qualsiasi dimensione e schiacciarlo per produrre un'impronta digitale di lunghezza fissa.

Per farlo funzionare però sono utilizzate delle funzioni dette COLLISION-RESISTANT, per abbassare quasi a 0% la probabilità che 2 messaggi diversi diano lo stesso Hash.

- Come funziona quindi la firma:

1. firmo elevando il plaintext alla mia chiave segreta d:

$$\text{(messaggio)} \xrightarrow{\text{PESANTE}} \text{Firma} \Rightarrow (\text{Hash(messaggio)})^d \rightarrow \text{Firma (efficiente)}$$

2. Invio:

$$(m, \text{Firma}) \rightarrow \text{questo è in chiaro (NO CONFIDENTIALITY)}$$

$$(m, \text{Firma})^{pk_B} \rightarrow \text{questo è cifrato (SI CONFIDENTIALITY)}$$

$(pk_B = e \text{ di Bob})$

3. Verifica:

il ricevente prende la mia chiave pubblica per spaccettare la firma:

$$\begin{array}{ccc} (\text{messaggio}, \text{Firma}) & & \\ \downarrow \text{fa Hash} & & \downarrow \\ \text{Hash(messaggio)} & \xrightarrow{\text{Se sono uguali, OK!}} & (\text{hash}(m))^d^{e_{\text{mia}}} = \text{Hash}(m) \end{array}$$

DSS (Digital Signature Standard)

Cos'è: È lo standard NIST (FIPS 186) per la firma digitale. È un documento/manuale delle regole.

A cosa serve: Serve solo a "firmare" i messaggi, non a cifrarli o a scambiare chiavi (a differenza di RSA).

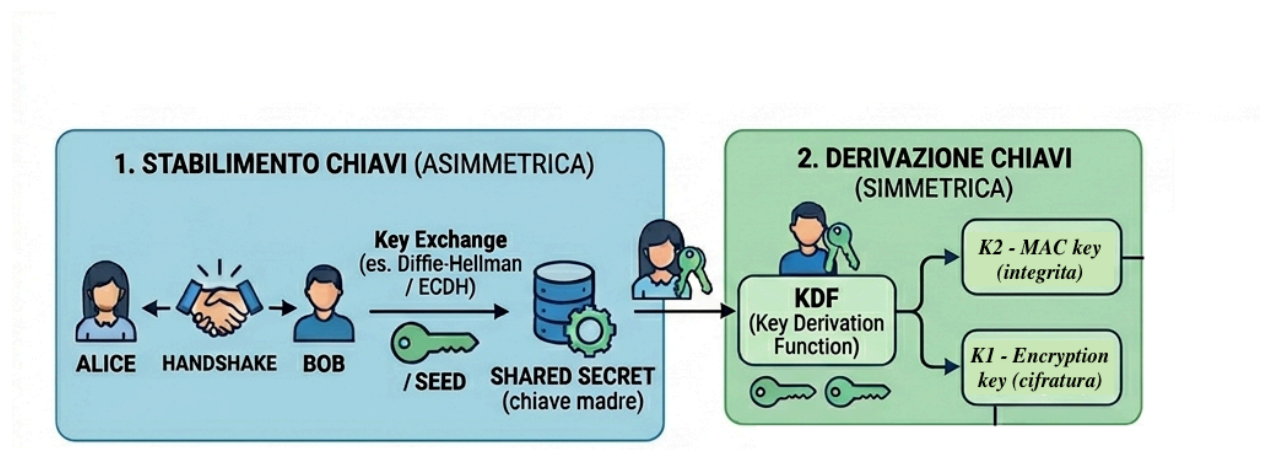
Cosa garantisce:

1. **Autenticità:** Sei sicuro di chi ha firmato.
2. **Integrità:** Il messaggio non è stato alterato.
3. **Non ripudio:** Chi firma non può negare di averlo fatto.

Algoritmi inclusi: Supporta principalmente **DSA**, **RSA** e **ECDSA** (Curve Ellittiche).

In breve: Il DSS è il "regolamento" ufficiale che dice come mettere una firma digitale valida su un documento.

COLLEGA GLI ARGOMENTI



4. PRG e CRITTOGRAFIA SIMMETRICA

Ora entriamo nel cuore della crittografia, parlando dell'idea Teorica dei generatori PRG e della loro traduzione nel mondo reale in algoritmi diventati fondamentali nella vita di tutto i giorni per proteggere i nostri dati, come l'AES.

CRITTOGRAFIA SIMMETRICA (storia)

Partiamo da un pò di storia, parleremo di cosa sta alla base di ogni algoritmo di cifratura simmetrico e per cosa viene utilizzato, iniziamo.

La Crittografia Simmetrica è una crittografia basata su un'unica chiave che deve essere condivisa tra 2 o più persone, orientata alla performance. Algoritmi come dicevo prima l'AES opera in modo meccanico a velocità elevatissime ma il difetto è proprio il problema del passarsi una chiave in modo sicuro, ed è il motivo per cui esiste la crittografia asimmetrica.

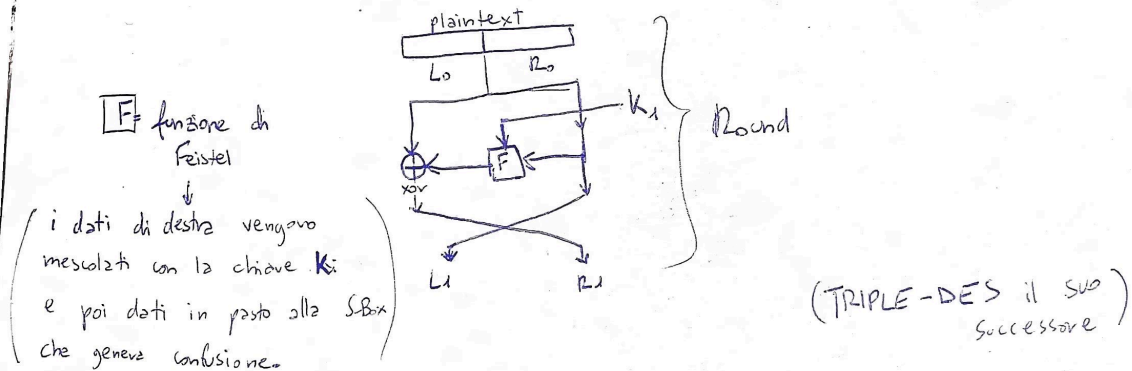
Tutto si basa su 2 regole inviolabili dettate da uno dei padri della crittografia, Claude Shannon:

- **CONFUSIONE**: l'algoritmo deve distruggere ogni legame logico tra la chiave segreta e il testo cifrato, solitamente tramite sostituzione (usando delle tabelle dette S-Box).
- **DIFFUSIONE**: l'algoritmo deve garantire che cambiando anche un solo bit di entrata, l'output dovrà propagare questa modifica in modo che almeno la metà dei bit di uscita cambi completamente.

Algoritmi Storici:

- DES (Data Encryption Standard) - Buato dopo 20 anni X

basato su un'architettura chiamata Rete di Feistel (16 Round)



- AES (Advanced Encryption Standard)

Si basa su una griglia matematica (SP- Network)

- Applica la confusione pura sostituendo ogni singolo byte (Sub Bytes)
- Applica la diffusione pura mescolando ~~orizzontalmente~~ orizzontalmente e verticalmente la griglia (Shift Rows e Mix Columns) in modo rapido e inattaccabile.

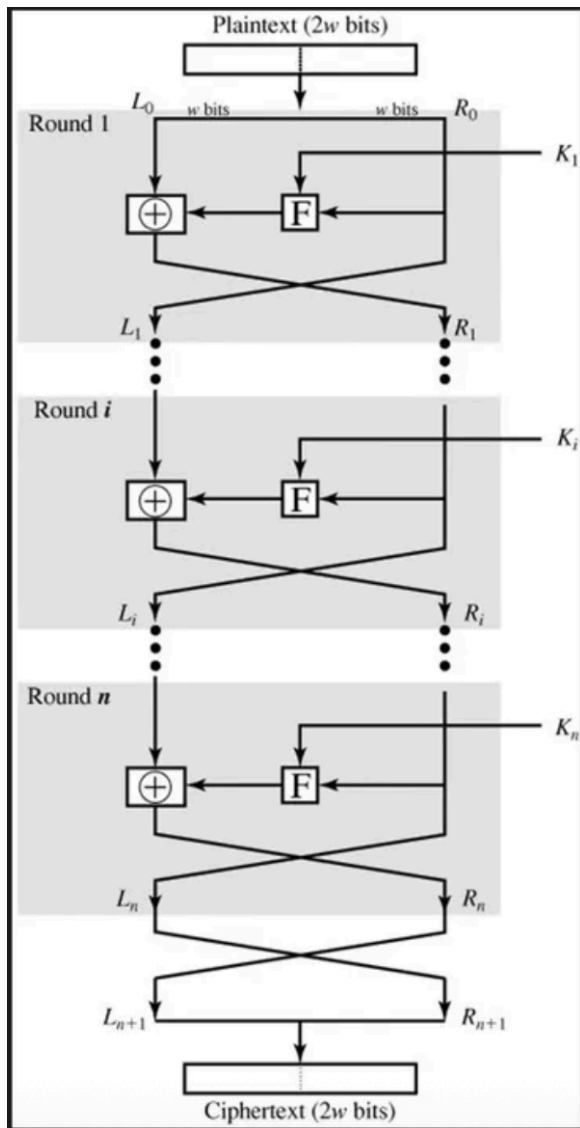
l'AES piega i dati in una matrice $n \times n$ (chiamata Stato), ad ogni ciclo (10 per 128 bit) fa passare questa matrice attraverso:

1. Sub Bytes (la confusione)
2. Shift Rows (inizio diffusione)
3. Mix Columns (motore diffusione)
4. Add Round Key (la maschera)

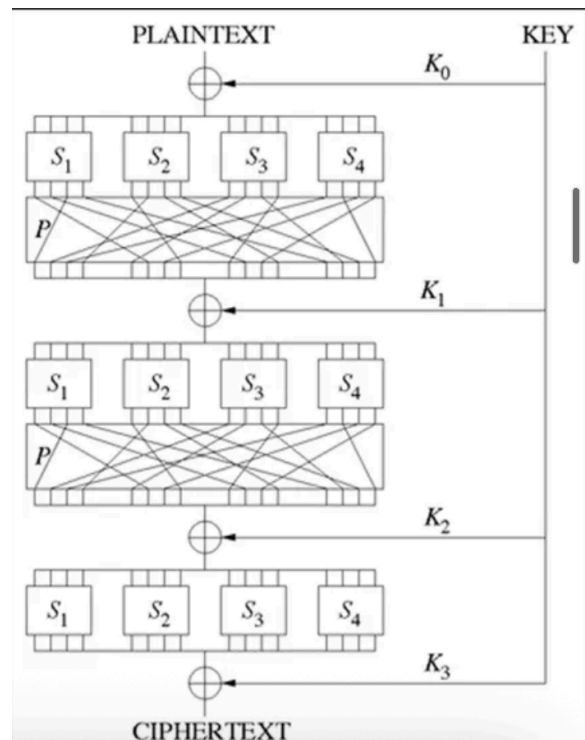
(l'ultimo giro si salta il 3)

APPROFONDIMENTO: DES vs AES

Ripartiamo dai concetti iniziali, ripartiamo da Claude Shannon e Feistel, PRG e HCP.



FEISTEL (DES)



SP - NETWORK (AES)

1. FEISTEL (a sinistra)

È quella che abbiamo visto finora (usata da algoritmi come DES o Camellia (TLS lo usa spesso)).

- **Come lavora:** la funzione **F** prende la metà destra, usa la chiave come istruzione e le **S-Box** per creare confusione. Quando il risultato "caotico" esce dalla funzione e colpisce la metà sinistra tramite lo **XOR**, l'operazione propaga quel cambiamento bit per bit: ogni singolo bit alterato dalla S-Box forza il bit

corrispondente della metà sinistra a invertirsi, "infettandola" matematicamente. (usa n sottochiavi quante i round)

- **Il vantaggio:** È molto comoda perché per decifrare basta fare i passaggi al contrario, usando la stessa identica struttura.

2. SPN (a destra - usata da AES)

Invece di dividere a metà, la SPN lavora su **tutto il blocco contemporaneamente**. Le lettere che vedi nell'immagine (S e P) sono esattamente quello che abbiamo spiegato prima:

- **S (Substitution):** È la **Confusione**. Prende i bit a pezzetti (S1, S2, S3, S4) e li sostituisce usando le S-Box.
- **P (Permutation):** È la **Diffusione**. Prende tutti i bit usciti dalle S-Box e li incrocia (quelle linee che si intrecciano nell'immagine), spalmandoli su tutto il blocco.

Qual è la differenza fondamentale?

- **In Feistel:** In ogni round modifichi solo **metà** del messaggio. Serve quindi un numero maggiore di round (giri) per raggiungere l'effetto valanga su tutto il blocco.
- **In SPN:** In ogni round modifichi **tutto** il messaggio in un colpo solo. Questo la rende potenzialmente più veloce e aggressiva nel creare confusione, ma richiede componenti matematiche più complesse per poter tornare indietro (decifrare).

PRG (Pseudo Random Generators)

Partiamo dal rispondere ad una domanda: Che cos'è una PRG??

Una PRG non è altro che una funzione che prende in ingresso una chiave (denominata SEED), che non è altro che la nostra chiave simmetrica Master Key che ci siamo passati prima tramite Diffie-Hellman per esempio, per generare una stringa di bit pseudo-casuali infinita.

In realtà non è esattamente quella la chiave utilizzata, di fatto per comodità si derivano chiavi secondarie per i vari compiti partendo dalla Master Key. Tramite una KDF (Key Derivation Functions), vengono generate dalle corrispettive parti (quindi sia da Alice che da Bob) delle chiavi derivate.

Dicevamo, la PRG, che segue l'idea del 'One-Time Pad' , prende in ingresso questa chiave derivata (seed) e lo frulla attraverso una formula matematica per sputare fuori una sequenza chilometrica di numeri (stringa). Questa lunghissima sequenza è "pseudo-casuale": per un hacker sembra totalmente imprevedibile (come una cascata di dadi lanciati a caso), ma in realtà se conoscesse il *seme* iniziale potrebbe ricreare l'esatta sequenza.

A cosa serve in pratica? Per i computer generare il caso "vero" (tipo misurare le variazioni di temperatura del processore) è un processo lentissimo. Il PRG risolve il problema: prende un pizzico di vera casualità e lo espande istantaneamente per creare tonnellate di numeri sicuri.

⇒ La traduzione nel mondo reale della PRG è per esempio l'AES visto prima che però ha dei limiti.

XOR - $\oplus$

Input A	Input B	Risultato ($A \oplus B$)
0	0	0
0	1	1
1	0	1
1	1	0

In crittografia lo XOR è l'unica operazione che garantisce la Perfect Secrecy se combinato con un bit casuale, ovvero il risultato finale avrà sempre probabilità $\frac{1}{2}$ di essere 1 e probabilità $\frac{1}{2}$ di essere 0.

$$\text{Testo} \oplus \text{Chiave} = \text{Cifrato} \rightarrow \text{Cifrato} \oplus \text{Chiave} = \text{Testo}$$

CIFRATURA SIMMETRICA A BLOCCHI

Fino a questo momento, ho omesso un particolare intenzionalmente perchè rendeva più intuitivo tutto il processo: ovvero che questa stringa infinita generata dalla PRG, va tagliata a blocchi ed eseguito lo XOR con il messaggio in chiaro anche lui diviso in blocchi della stessa dimensione. (Proprio per rispettare l'algoritmo di One-Time-Pad).

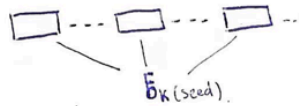
Questo ci porta ad un problema non indifferente della crittografia simmetrica, ovvero che è a blocchi!

Non esiste una funzione che macini tutto quanto il messaggio in un colpo solo, ed è proprio questo il limite dell'AES a cui mi riferivo prima, ovvero che l'algoritmo lavora su blocchi di bit molto piccoli alla volta e questo crea dei problemi di sicurezza.

CIFRATURA A BLOCCHI

3 Modi:

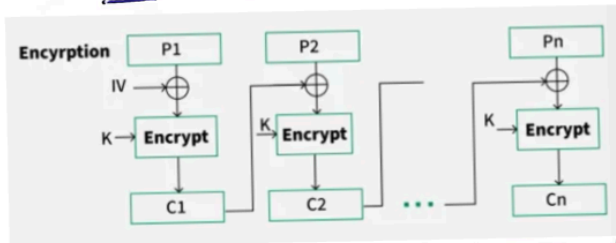
- ECB (Electronic Code Block):



(PRO: parallelizzabile (veloce))
(CONTRA: malleabile. $\begin{smallmatrix} \square & \square & \square \\ 1 & 3 & 2 \end{smallmatrix}$)

Ogni blocco di testo in chiaro viene cifrato in modo completamente indipendente dagli altri, usando sempre la stessa chiave.

- CBC (Cipher Block Chain):

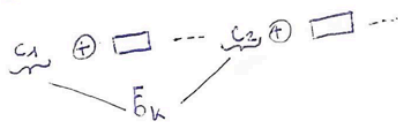


(PRO: Sicuro)
(CONTRA: Non parallelizzabile (lento))
* se ho un errore su un blocco, salta tutto.

Risolve la debolezza di ECB concatenando i blocchi. Ogni blocco, prima di essere cifrato, viene XORato con il risultato cifrato del blocco precedente. Il primo blocco viene XORato con IV (Initialization Vector).

- CTR (Counter):

$C = \text{counter}$



(PRO: sicuro
* parallelizzabile)
(CONTRA: /)

Invece di cifrare il testo, l'algoritmo cifra un semplice contatore condiviso tra le parti (Alice e Bob), che viene incrementato ad ogni blocco.

Il contatore appena cifrato viene poi XORato col blocco di testo in chiaro corrispondente.

QUINDI: l'algoritmo spezza il messaggio in blocchi, spezza la stringa generata casuale (PRG) in blocchi e fa lo XOR tra blocchi usando ECB/CBC/CTR.

Alla fine ricompatta tutto in un unico blocco cifrato finale!

APPROFONDIMENTO (CURIOSITÀ):

Esiste un cifrario a blocchi moderno usato comunemente con l'**AES-XTS** (XEX Tweakable Block Cipher with Ciphertext Stealing).

AES-XTS (Full Disk Encryption)

L'**AES-XTS** è lo standard industriale per la crittografia degli interi dischi rigidi (usato da BitLocker, FileVault, LUKS). Nasce per risolvere due vincoli fisici insormontabili degli hard disk:

1. **Dimensione Fissa:** I settori del disco (es. 4 KB) non hanno spazio extra. Se entrano 4 KB in chiaro, devono uscire 4 KB cifrati. Non si possono aggiungere metadati (come IV o Nonce).
2. **Accesso Casuale Rapido:** Il processore deve poter decifrare istantaneamente il settore 1.000.000 senza dover scorrere i precedenti.

Il Rischio: Senza metadati, usare un AES standard produrrebbe un difetto fatale: salvare lo stesso identico file (es. blocchi di zeri) in settori diversi genererebbe lo stesso output cifrato, rivelando la struttura del disco agli hacker.

La Soluzione: La Doppia Chiave e il "Tweak"

L'AES-XTS risolve il problema ancorando la crittografia alla *posizione fisica* del dato. Divide la chiave principale in due sotto-chiavi:

- **Chiave 1 (Generazione del Tweak):** Prende l'indirizzo fisico del settore sul disco (LBA) e lo cifra. Il risultato è il **Tweak**, una variabile matematica unica per quel preciso settore.
- **Chiave 2 (Cifratura Dati):** Esegue l'algoritmo AES standard.

Il Meccanismo (XOR-Encrypt-XOR)

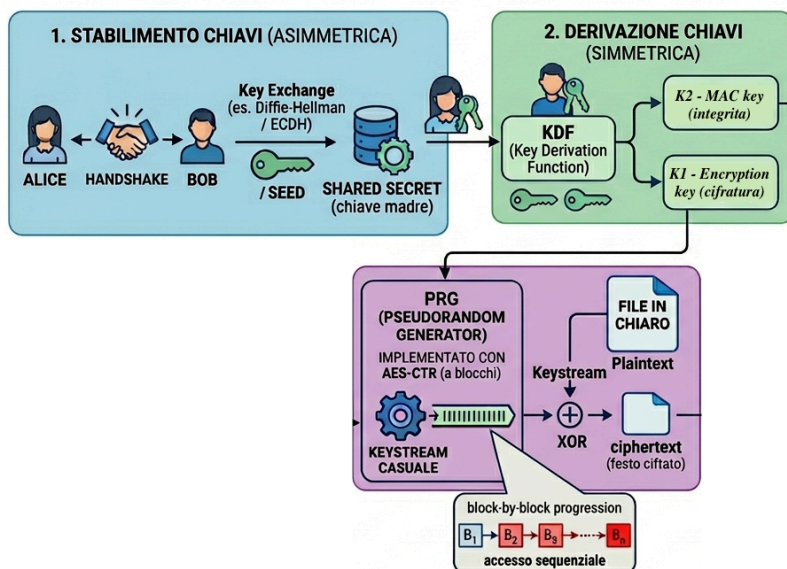
Il processo avviene in tre micro-fasi esatte:

1. Il dato in chiaro viene fuso matematicamente (XOR) con il Tweak.
2. Il blocco risultante subisce la cifratura AES tramite la Chiave 2.
3. L'output cifrato viene fuso nuovamente (XOR) con il Tweak.

Risultato Architeturale:

Poiché l'equazione include l'indirizzo fisico, scrivere lo stesso identico file sul Settore 10 e sul Settore 500 produrrà due ciphertext **radicalmente diversi**. Il sistema garantisce accesso casuale istantaneo e azzeri i pattern visibili, senza aggiungere un singolo byte di peso.

COLLEGA GLI ARGOMENTI



Abbiamo visto come scambiarsi una master key, che dopo una KDF diventa il nostro seme (SEED) con cui attivare l'algoritmo di cifratura simmetrico, ovvero una PRG come AES che per ovviare al problema di cifrare un pezzo alla volta utilizza il CTR durante il processo e ottiene come risultato un unico blocco finale che è proprio il nostro messaggio CIFRATO!

C'è un problema ora: come sappiamo se la nostra PRG è sicura?

5. SICUREZZA DI UNA PRG

Ecco la spiegazione rigorosa di come tre concetti più teorici si legano per provare che un Generatore Pseudo-Casuale (PRG) è sicuro.

1) TEOREMA DI YAO (NEXT-BIT TEST)

Nella crittografia moderna, non possiamo accontentarci di numeri che "sembrano" casuali. Ci serve una prova formale. Il Teorema di Yao fornisce esattamente questo: una definizione matematica di cosa renda un PRG crittograficamente sicuro.

Il concetto:

Yao introduce il Next-Bit Test (Test del bit successivo). Immagina che un avversario (un algoritmo con una potenza di calcolo realistica) osservi i primi n bit generati da uno PRG.

Il test stabilisce che il generatore è sicuro se e solo se l'avversario non ha alcun metodo per prevedere il bit $n + 1$, con una probabilità significativamente superiore al 50%.

La validità pratica:

Yao ha dimostrato matematicamente che se una sequenza supera questo test, è "computazionalmente indistinguibile" da una sequenza veramente casuale.

In pratica: se non puoi prevedere il bit successivo, l'intera stringa è crittograficamente solida.

La sfida ingegneristica diventa quindi una sola: come generiamo una serie di bit dove il successivo è sempre imprevedibile?

2) HCP (HARD-CORE PREDICATE)

Per superare il test di Yao, ci serve una fonte di imprevedibilità garantita. Qui entra in gioco l'HCP (Hard-Core Predicate).

Il concetto:

In crittografia lavoriamo con le **Funzioni One-Way** (funzioni unidirezionali),

operazioni matematiche facilissime da calcolare in un senso, ma impossibili da invertire (es. la moltiplicazione di due numeri primi giganti è facile, la fattorizzazione del risultato è difficilissima).

L'HCP è **un singolo bit di informazione** associato all'input di questa funzione.

La proprietà fondamentale:

La teoria dell'HCP dimostra che, conoscendo solo l'output della funzione, indovinare il valore di quello specifico bit (l'HCP) è matematicamente complesso **esattamente quanto lo è invertire l'intera funzione unidirezionale**.

L'impatto pratico:

Abbiamo appena trovato la nostra "ancora di sicurezza". Se basiamo la sicurezza del nostro sistema su un problema matematico che richiede milioni di anni per essere risolto, l'HCP ci garantisce che indovinare quel singolo bit richiederà lo stesso identico tempo e sforzo computazionale.

3) GENERATORE DI GOLDWASSER-MICALI (BBS)

Ora abbiamo la regola (Yao) e il materiale grezzo (l'HCP). L'architettura logica che li unisce serve a costruire un PRG dove la sicurezza non è un'opinione, ma una certezza matematica legata alla **difficoltà della fattorizzazione**.

Il concetto:

Blum e Micali hanno progettato un sistema iterativo. Il loro algoritmo utilizza il problema dei Residui Quadratici per generare il bit HCP successivo da mettere nella stringa.

Come opera l'algoritmo:

1. **Inizializzazione:** Si parte da un valore segreto iniziale (il *seed* o seme), che chiamiamo x_0 .
2. **Estrazione e Iterazione:** Il sistema calcola il valore successivo x_1 applicando la funzione unidirezionale a x_0 . Contemporaneamente, **estrae l'HCP** del valore corrente. Questo HCP diventa il primo bit ufficiale della stringa in output.
3. **Il Loop:** Il processo si ripete. x_1 viene usato per calcolare x_2 , estraendo un nuovo HCP (il secondo bit dell'output). E così via.

La sintesi logica:

Poiché ogni singolo bit generato è un Hard-Core Predicate protetto dalla difficoltà di fattorizzare numeri giganti, la sua imprevedibilità è garantita. Nessun hacker può indovinare se il bit successivo sarà 0 o 1 con una probabilità migliore del lancio di una moneta.

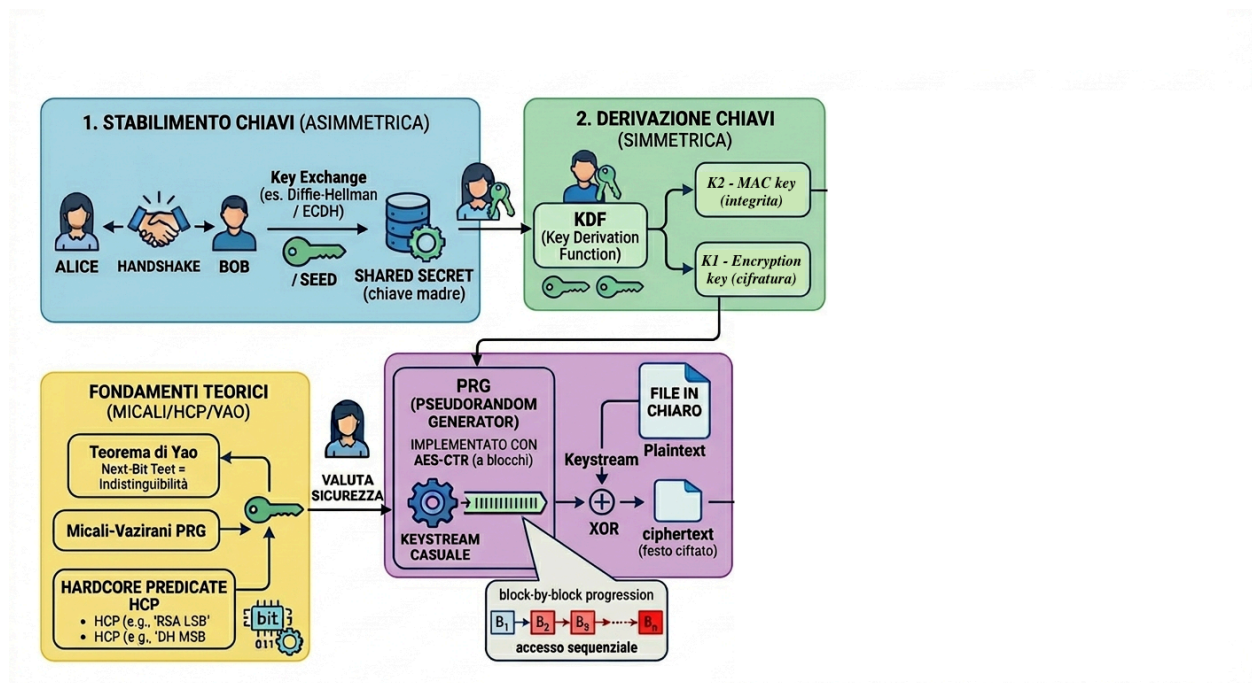
Essendo ogni bit singolarmente imprevedibile, la stringa infinita generata **soddisfa rigorosamente il Next-Bit Test di Yao**. Abbiamo così ottenuto un PRG di cui possiamo dimostrare l'assoluta sicurezza computazionale.

(Nota): Il meccanismo è identico per il Blum-Micali classico (Logaritmo Discreto) e per il Blum-Blum-Shub/Goldwasser-Micali (Falsi Quadrati). Cambia solo la funzione unidirezionale usata.

Nella sua teoria Micali utilizza la figura del Distinguisher per dimostrare per assurdo che un attaccante vedendo cambiare da un messaggio₁ cifrato a un messaggio₂ cifrato un bit alla volta, non potrebbe che avere il 50% di probabilità di indovinare e non un punto in più.

Se avesse anche solo il 51%, utilizzando il limite di Chernoff raggiungerebbe il 100% di accuratezza facendo saltare la sicurezza della stringa.

COLLEGA GLI ARGOMENTI



Con questo capitolo ci siamo fermati e abbiamo realizzato una cosa importante: Possiamo **AFFERMARE** che la nostra PRG è sicura e sappiamo il perchè grazie a Yao e Micali.

Ora il passo successivo è un altro: garantire che nessuno manometta il pacchetto durante il viaggio!

Approfondimento: Dalla Teoria alla Realtà (CSPRNG)

Per comprendere come i sistemi informatici generino numeri casuali sicuri (alla base di chiavi RSA, Wi-Fi o TLS), è necessario distinguere tra il modello matematico teorico (perfetto ma lento) e l'ingegneria del mondo reale (veloce e sicura).

Questi sistemi prendono il nome di **CSPRNG** (*Cryptographically Secure Pseudo-Random Number Generators*).

BASE TEORICA (ripasso pre-spiegazione)

Come già visto nella parte di Crittografia, l'idea teorica alla base della generazione casuale sicura sfrutta problemi matematici complessi (come la fattorizzazione di grandi numeri o il logaritmo discreto) abbinati al concetto di **HCP (Hard-Core Predicate)**.

Come prima, l'algoritmo teorico più celebre basato su questa idea è il **Blum Blum Shub (BBS)**.

Funziona così:

1. **Inizializzazione:** Si sceglie un grande numero N (prodotto di due numeri primi, come in RSA) e un valore segreto di partenza (*Seed*).

2. **La Funzione Unidirezionale:** Ad ogni ciclo, il sistema esegue un calcolo facile in una direzione ma impossibile da invertire senza conoscere i fattori (es. l'elevamento al quadrato modulo N).

3. **Estrazione del Bit (HCP):** Dal risultato di questo calcolo, il sistema estrae un solo bit (ad esempio, l'ultimo bit, o bit meno significativo). La matematica dimostra che indovinare questo singolo bit estratto equivale a risolvere l'intero problema della fattorizzazione.

4. **Iterazione:** Il risultato viene re-inserito nella funzione per estrarre il bit successivo, creando una stringa infinita.

Il problema: Questo approccio offre una sicurezza matematica assoluta (dimostrabile), ma è computazionalmente insostenibile. Richiede calcoli su numeri di migliaia di bit solo per produrre un singolo "1" o "0". Se un server dovesse generare megabyte di chiavi al secondo usando il BBS, collasserebbe.

1. SOLUZIONE REALE (Architettura dei CSPRNG moderni)

L'obiettivo di un CSPRNG è trasformare una piccola quantità di rumore fisico (entropia) in un flusso infinito e matematicamente imprevedibile di bit. Da questo flusso, il sistema si limiterà a "tagliare" dei blocchi della lunghezza che gli servono, ogni volta che un protocollo richiederà la creazione di una nuova chiave.

Fase A: La Raccolta dell'Entropia (Il Seme Iniziale)

Poiché il computer non può generare casualità dal nulla, il Sistema Operativo utilizza un "raccolgitore di entropia" che campiona eventi fisici misurabili ma crittograficamente imprevedibili dell'hardware:

- I millisecondi di latenza tra gli input della tastiera o i movimenti del mouse.
- Il rumore termico e le fluttuazioni microscopiche di voltaggio della CPU.
- I tempi esatti di accesso ai settori dei dischi di memoria.

Questi dati ambientali vengono processati (tipicamente tramite funzioni di Hash) per generare un **Seme (Seed)** iniziale denso e puramente casuale (es. 256 bit).

Fase B: L'Espansione tramite AES-CTR

Il Seme fisico iniziale si genera lentamente. Per fornire ai protocolli di rete gigabyte di numeri casuali senza latenza, il sistema utilizza l'algoritmo AES.

In questo contesto, l'AES non viene usato per garantire la riservatezza di un messaggio, ma come Funzione Pseudo-Casuale (PRF):

1. **Inizializzazione:** Il sistema usa il Seme fisico appena raccolto come Chiave Segreta dell'algoritmo AES.
2. **Input Sequenziale:** Prende un contatore numerico standard (il numero **1**) e applica un padding di zeri per raggiungere la dimensione di blocco fissa di 128 bit richiesta dall'AES (**0000...00000001**).
3. **L'Elaborazione:** L'AES elabora questo blocco fondendolo con la chiave segreta attraverso 14 round di sostituzioni e permutazioni non lineari (tramite le S-Box). L'output risultante è un blocco di 128 bit crittograficamente non correlato all'input.
4. **L'Effetto Valanga:** Il sistema inserisce nel calcolo il contatore successivo, il **2** (**0000...00000010**). Sebbene differisca dal precedente per un singolo bit, l'architettura matematica dell'AES innesca un "effetto valanga": l'output a 128 bit sarà radicalmente diverso e imprevedibile rispetto al blocco precedente.
5. **Velocità Hardware:** Iterando il contatore (**3, 4, 5...**), il processore genera un

flusso continuo di bit pseudo-casuali. Sfruttando i set di istruzioni hardware dedicati all'AES presenti nelle CPU moderne (AES-NI), il sistema produce miliardi di bit sicuri al secondo senza saturare le risorse di calcolo.

2. APPLICAZIONE (Generazione delle chiavi RSA)

Questo flusso continuo di bit pseudo-casuali viene utilizzato dai protocolli di sicurezza per istanziare parametri crittografici. Nel caso dell'RSA, l'obiettivo iniziale è individuare due numeri primi sufficientemente grandi (p e q).

1. **Estrazione:** Il sistema preleva dal flusso generato dal CSPRNG un blocco di bit della lunghezza necessaria (es. 2048 bit continui).

2. **Formattazione del Candidato:** Per ottimizzare i tempi di calcolo, il sistema manipola i margini del blocco estratto: forza l'ultimo bit a 1 (garantendo che il numero sia dispari, poiché un numero pari non può essere primo) e forza il primo bit a 1 (garantendo che il numero abbia effettivamente una magnitudo di 2048 bit). Si ottiene così il "Candidato P".

3. Test di Miller-Rabin e Loop di Scarto

Per la legge matematica della densità dei numeri primi, a grandezze crittografiche (es. 2048 bit) la probabilità di estrarre casualmente un numero primo è bassissima (circa 1 su 700). Poiché il fallimento è l'esito statisticamente quasi certo, l'ingegneria informatica non cerca una formula magica per generare il numero perfetto al primo colpo, ma compensa questa rarità matematica con la pura velocità hardware, procedendo per scarti rapidissimi.

Il sistema agisce in tandem:

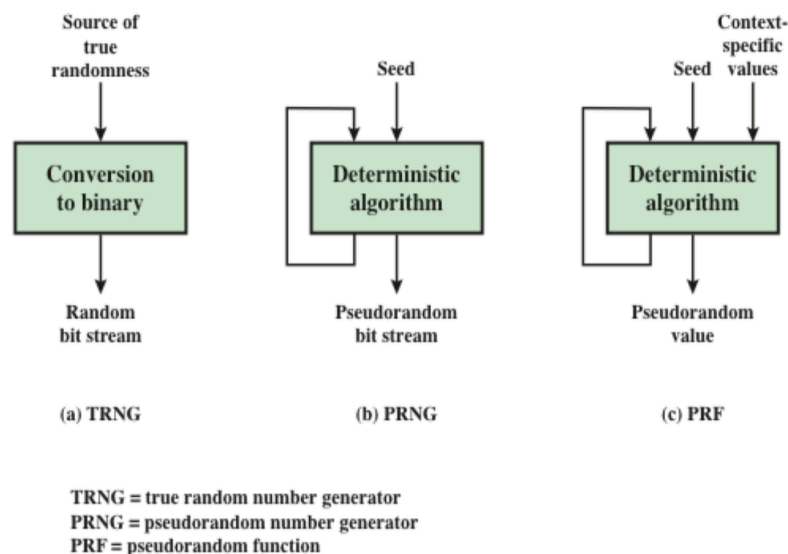
- **Il Filtro (Test di Miller-Rabin):** Il "Candidato P" subisce il test probabilistico di Miller-Rabin. Invece di tentare una lentissima fattorizzazione, questo algoritmo esegue prove statistiche mirate a "bocciare" i numeri composti in pochi microsecondi. Alla prima iterazione fallita, il candidato è scartato. Solo se le supera tutte, la probabilità che sia primo sfiora il 100% (es. 99,9999%).
- **Il Loop (Forza Bruta):** Nel millisecondo esatto in cui un candidato viene bocciato, il sistema taglia istantaneamente un nuovo segmento di 2048 bit dal flusso inesauribile del CSPRNG, lo formatta e riavvia il test.

Grazie a questo ciclo continuo, il processore testa e scarta migliaia di candidati al secondo finché non "fa centro". Il numero validato diventa ufficialmente il parametro **P**. A quel punto, il loop si azzerava e la spietata selezione riparte da capo per individuare **Q**, fornendo infine i due pilastri per generare l'intera chiave RSA.

PRF (Pseudo Random Functions)

la fase di applicazione di cui parlavamo prima tagliando la stringa generata, si traduce nell'utilizzo dei PRF.

A differenza del PRNG (che genera un flusso continuo e generico), il PRF prende il Seed e lo fonde con dei valori di contesto, quali possono essere un User ID o un App ID.



le 3 fasi del CSPRNG (entropia-PRG-applicazione)

6. INTEGRITY DEL CIPHERTEXT

Per garantire che il nostro messaggio cifrato (ciphertext) non venga manomesso durante il suo viaggio verso Bob, utilizziamo le Pseudo Random Functions, un concetto simile alle PRG di prima.

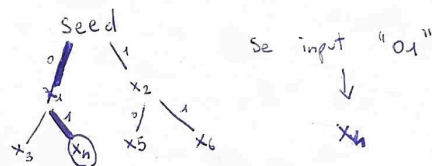
PRF (PSEUDO RANDOM FUNCTIONS) e MAC

PRF (Pseudo Random Functions) e MAC

Le PRF differenziamo dalle PRG per una caratteristica particolare:

Sono ad accesso casuale, mentre le PRG sono ad accesso sequenziale.

- STRUTTURA DI UNA PRF: ha una struttura ad albero di nome GGM,



- MAC, cos'è?

Il mac è come noi decidiamo di usare quel motore (PRF) per proteggere il file.

È la sua realizzazione pratica principale, come poteva essere AES per PRG.

- come è fatta la funzione:

$$\text{MAC} = \text{PRF}(\text{Key}, \text{Ciphertext})$$

- Come funziona:

Forziamo la PRF a lavorare a "catena" usando il CBC-MAC.

Questo ci dà un blocco finale molto sicuro che dipende da ~~tutti i suoi blocchi precedenti~~ tutti i suoi blocchi precedenti.

(cipher-block-chaining)

Un'alternativa al CBC-MAC è l'utilizzo del HMAC (sfrutta l'hash).

diretta e veloce.
utile per calcolare ~~cifatura~~ ~~decifatura~~
particolare quando i pacchetti diversi
arrivano sfasati di ordine da
internet. (La PRG si bloccherebbe).
(es: decifra un preciso
pacchetto di un video)

HMAC

sfrutta la potenza e velocità delle funzioni HASH (come SHA-256)

- PROBLEMA: se creo un Hash ("chiave condivisa" + messaggio), sono vulnerabile ad un Length Extension Attack. ~~Unsecure~~

- SOLUZIONE: l'HMAC fa un doppio passaggio:

$$\text{HMAC}(k, m) = \text{Hash}((k' \oplus \text{opad}) \parallel \text{Hash}((k' \oplus \text{ipad}) \parallel m))$$

Con:

- $k' = \text{Hash}(k)$
- opad = outer pad (0x5c per dm. blocco)
- ipad = inner pad (0x36 per dm. blocco)
- Hash

La differenza strutturale: PRNG vs. PRF

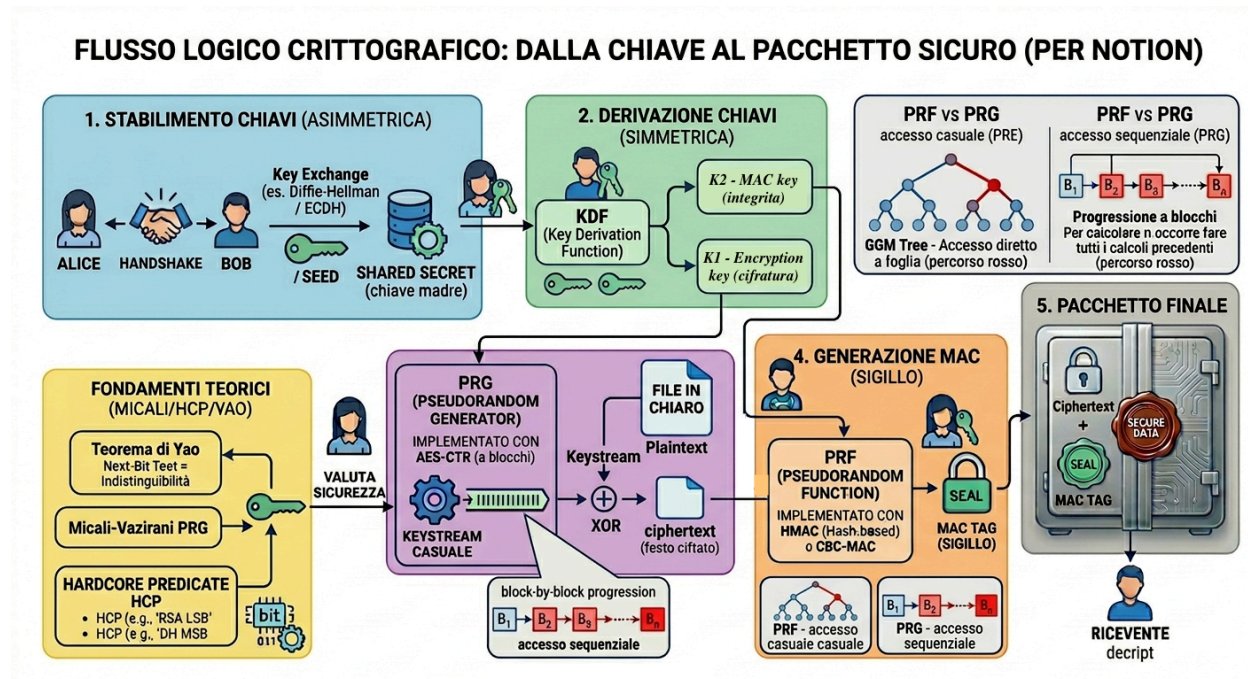
Per capire la PRF, dobbiamo contrapporla al PRNG che abbiamo studiato finora.

- **Il PRNG ha una "memoria" (Stateful)**: Come abbiamo visto con l'AES-CTR, il PRNG si avvia con un Seme e poi mantiene un contatore interno (1, 2, 3...). Ogni volta che gli chiedi dei bit, lui avanza e ti dà bit nuovi. Se perdi il conto di dove è arrivato, non puoi ricostruire il flusso. Serve per generare materiale *nuovo e indipendente*.
- **La PRF è "senza memoria" (Stateless)**: Non ha contatori che avanzano da soli. Non genera nulla finché non le dai i parametri. Serve per generare materiale *riproducibile e mirato*. Se due computer diversi (es. il tuo PC e il server di Google) inseriscono gli stessi dati nella stessa PRF, ottengono lo stesso risultato.

A differenza del PRNG (che genera un flusso continuo e generico), il PRF prende il Seed e lo fonde con dei valori di contesto, quali possono essere un User ID o un

App ID.

COLLEGA GLI ARGOMENTI



Siamo arrivati alla fine del percorso, abbiamo visto come la crittografia asimmetrica ci permette di scambiare una chiave master key, da cui deriviamo una chiave che utilizziamo per cifrare i nostri dati tramite una PRG (AES-CTR per esempio) sicura grazie alle fondamenta teoriche di Yao e Micali.

La stringa a blocchi viene XORata al Plaintext per ottenere il Ciphertext a cui aggiungiamo il timbro inserendo nel messaggio un MAC finale (CBC-MAC per esempio).

Finalmente abbiamo un messaggio sicuro e impossibile da manomettere pronto per essere spedito a Bob!

7. USI "CREATIVI" E UTILI DELLA CRITTOGRAFIA

OBLIVIOUS TRANSFER

(Il Lancio della moneta)

Il lancio della moneta è un problema di 2 persone (Alice e Bob) che devono prendere una decisione binaria a distanza (testa o croce) senza potersi vedere e non fidandosi l'uno dell'altro.

La soluzione sfrutta il cuore dell'algoritmo di Rabin:

- Se per un Z_p^* valeva $\phi(p)/\text{eulero} \Rightarrow$ la metà sono quadrati perché ogni numero ha 2 radici
- per un $n = p \times q$ $Z_n \Rightarrow$ abbiamo 4 radici, 2 per p , 2 per q .

$\Rightarrow$ È SOLO CHI HA p e q PUÒ CALCOLARE TUTTE!

- COME FUNZIONA:

- 1) Alice crea $n = p \times q$ e lo invia a Bob ($\text{mod } n$)
 $p \times q$
- 2) Bob sceglie un x a caso (secret), lo eleva al quadrato e lo invia
($y = x^2 \text{ (mod } n)$)
 $\hookrightarrow$ crea un quadrato.
- 3) Alice, che conosce p e q , calcola le 4 radici di quel numero y , ne sceglie una a caso z e la rispedisce a Bob.

h) Il verdetto:

- se Alice invia lo stesso numero scelto da Bob ~~o~~ o il suo negativo ($\pm x$), Bob non ha imparato nulla, Alice vince!
- se Alice invia una delle altre 2 radici, Bob può usare il MCD per scoprire i numeri p e q di Alice. Bob vince!

👉 questo concetto è alla base di Oblivious Transfer (Trasferimento Obliquo):
"io ti invio delle informazioni, ma io mittente rimango all'oscuro (oblivious) di quale informazione tu sia riuscito effettivamente ad ottenere".

$\Rightarrow$ Quindi usiamo Oblivious Transfer per generare un evento casuale equo al 50% in cui né Alice né Bob decidono il risultato.

(aggiunta al perchè se Bob conosce 2 radici diverse allora vince: Perché la differenza matematica tra queste due radici $(x - y)$ condivide un divisore in comune con n .

Calcolando un semplice Massimo Comune Divisore (MCD) tra questa differenza e n , estrai all'istante il fattore segreto p !)

BIT COMMITMENT

Se prima generavamo un evento equo casuale (Obvious Transfer), ora l'obiettivo del bit commitment è invece quello di creare una promessa non modificabile.

Ovvero Alice sceglie b e viene vincolata irrevocabilmente a un bit $(0, 1)$ al tempo T_1 , fornendo a Bob la prova matematica per verificarlo solo al tempo T_2 .

- DEVE SODDISFARE 2 PROPRIETÀ:

1. Hiding: Bob riceve un output c che è computazionalmente impossibile da invertire e tornare al bit di partenza.
2. Binding: Alice invia l'output c , deve esserle impossibile al tempo T_2 fornire dei parametri per cambiare la scelta del bit iniziale, facendolo combaciare al primo output c .

- COME FUNZIONA:

Fase 1, Commit:

- 1) Alice sceglie il suo bit $b \in \{0, 1\}$
- 2) Genera un nonce r da b (evita che Bob bruteforzi l'hash di 0 e 1)

NONCE = r è un valore numerico univoco e non ripetibile utilizzato per rendere non deterministica la cifratura: garantisce che lo stesso messaggio, criptato più volte, produca output differenti, impedendo attacchi di REPLAY e analisi statistiche

- 3) Alice genera l'Hash: $c = \text{Hash}(b || r)$ e lo invia a Bob

Fase 2, Reveal:

- 1) al tempo T_2 , Alice invia a Bob il bit b e il nonce IN CHIARO.
- 2) Bob ricalcòla $c_2 = \text{Hash}(b || r)$ e lo confronta con il primo c , se combaciano è l'originale altrimenti scarta.

SECRET SHARING di SHAMIR (la "S" in RSA)

- L'IDEA:

Hai una chiave crittografica (il segreto S) che apre qualcosa di importante.
Voi distribuire questa chiave a n direttori/persone.

Rules:

- 1) Scegli una soglia t (compresa), ~~sempre~~ ^{che rappresenta} il numero minimo di persone che devono esserci per costruire la chiave (es: $n/2$)
- 2) Sotto quella soglia t , deve esserci lo 0% di probabilità di calcolarla.

- COME FUNZIONA?

Adi Shamir utilizza i Polinomi sfruttandone un loro principio geometrico:

- Per definire una Retta (grado 1), ti servono ~~esattamente~~ esattamente 2 punti, se ne hai 1, trovi infinite rette passanti per quel punto.
 - Ugualmente con la parabola (grado 2), servono esattamente 3 punti, meno $\rightarrow$ infinite parabole
- $\Rightarrow$ per un polinomio di grado $t-1$, servono almeno t punti.

- 1) Scegliamo una soglia t (es $t=3$)
- 2) Prendiamo il segreto S e lo rendiamo l'intercetta di un piano cartesiano ($x=0, y=S$)
- 3) Il computer calcola 5 punti casuali lungo (in questo caso) la parabola e li distribuisce alle n persone ($x=1, x=2, \text{etc...}$) e poi cancella la parabola dalla memoria

$\Rightarrow$ se si ritrovano solo 2 persone, $2 < t \Rightarrow$ Non sanno niente!
se le persone sono $\geq t \Rightarrow$ viene calcolata la parabola, che è unica al mondo per quei 3 punti e quindi -- la chiave.

I CRITTOGRAFI MANGIATORI (Dining Cryptographers)

David Chaum, 1988

(parlare in una stanza senza che nessuno capisca da quale bocca esce la voce)

- Scopo: Realizzare una rete di comunicazione che consenta il broadcast di un messaggio garantendo l'anonimato incondizionato del mittente.

- DEFINIZIONE DEL PROBLEMA:

Un gruppo di n crittografi (es: $n=3$, Alice, Bob, Charlie) condivide una cena.

Il cameriere informa il gruppo che il conto è già stato saldato.

Possibili paganti:

- Un'entità esterna (esdatore di lavoro)
- Esattamente uno dei crittografi seduti al tavolo.

Obiettivo: Determinare se il pagatore appartiene al gruppo, preservando ~~l'anonimato~~ l'assoluto anonimato del soggetto pagante nei confronti di tutti gli altri partecipanti e di eventuali osservatori esterni.

FASE 1:

Ogni coppia di crittografi adiacenti stabilisce un segreto b condiviso casuale.

- b_1 : Alice, Bob
- b_2 : Bob, Charlie
- b_3 : Charlie, Alice

FASE 2:

Ogni crittografo fa una dichiarazione D_i facendo la XOR tra il b alla sua sinistra con il b alla sua destra:

- SE IL CRITTOGRAFO NON HA PAGATO: dice la verità: $b_{sin} \oplus b_{dest}$
- SE HA PAGATO, mente (nega): $\neg(b_{sin} \oplus b_{dest}) \equiv b_{sin} \oplus b_{dest} \oplus 1$

FASE 3:

$$Result = D_A \oplus D_B \oplus D_C$$

CONCLUSIONI

- Se nessuno ha "negato" $\rightarrow$ ha pagato l'entità esterna $\Rightarrow R = 0$
- Se un crittografo ha "negato" $\rightarrow$ ha pagato un membro del gruppo $\Rightarrow R = 1$

DIMOSTRAZIONI A CONOSCENZA ZERO (ZERO KNOWLEDGE PROOFS - ZKP)

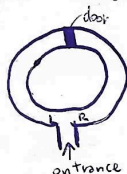
Goldwasser, Micali, Rackoff, anni '80

- IL PROBLEMA: Come fai a dimostrare a un server che conosci la tua password, senza mai inviarla?

(Se la invii (anche cifrata), il server la riceve, la elabora e potrebbe salvarla. Noi vogliamo convincerlo passandogli zero informazioni su quale sia il segreto.)

- LA SOLUZIONE:

I crittografi usano la metafora della caverna, un anello con un bivio iniziale e una porta magica in fondo che si apre solo con la parola d'ordine segreta.



Attori: Prover, P
Verifier, V

- COME FUNZIONA:

- 1) V resta fuori dalla caverna di spalle
- 2) P entra e a caso sceglie di andare a Left (L) o Right (R) e si ferma in fondo davanti alla porta magica
- 3) V si affaccia e le urla una sfida a caso (es: "P, esci dal percorso R") ("P, esci dal percorso L")
- 4) Se P conosce la parola magica, vince sempre. Se P è impostore, ha solo il 50% di indovinare da quale lato dovrà uscire.

- PERCHÉ FUNZIONA:

V ripete il gioco n volte di fila, la probabilità che un impostore indovini la scelta di V per n volte è $\frac{1}{2^n}$ (se $n=20$, prob = $\frac{1}{1\text{ milione}}$)

dopo gli n round, V è sicuro che P conosce il segreto senza aver mai sentito o visto P usare la parola magica.

- PROPRIETÀ CHE LA ZKP DEVE SODDISFARRE:

1. Completeness: se P sa il segreto, vince con prob. 100%
2. Soundness: se P mente, l'errore crolla esponenzialmente ad ogni iterazione
3. Zero-Knowledge: se l'affermazione è vera, nessun Verifier (anche disonesto) apprende nulla oltre al fatto che l'affermazione è vera!
